

Dwarpal design

This is the design of `@briqvent/dwarpal`, contract version **0.5**. It describes what the package does, the guarantees it makes and the rules an integration must follow. For step-by-step integration read the [user manual](#). For the SQL design, sequence diagrams and failure analysis read the [RBAC low-level design](#).

Status: the design is frozen at contract 0.5. The package is not released yet and nothing is published to npm. Where this document and the released code disagree, the released code and its changelog take precedence.

1. Purpose and scope

Dwarpal is an authentication and authorization kit on **Supabase Auth**. It is meant for websites with a static or server-rendered front end and, optionally, a Node.js back end. It provides:

- email and password sign-up with verification, sign-in, sign-out, forgotten and reset password, Google sign-in and TOTP multi-factor authentication;
- brandable default screens for those flows, or a headless controller for your own pages;
- a server library that turns a request's bearer token into a verified **Principal**: who the user is, which roles they hold for your site and which permissions those roles grant;
- SQL migrations that store clients, roles, permissions, enrollments and memberships, written only through audited SQL functions;
- optional helper functions for your own Row Level Security policies;
- an operator command-line tool, `auth-kit`, for migrations, client registration, the permission model, the first manager and diagnostics.

It is **not** a service, a row-level authorization engine, an admin console, a store for your business data, or a cookie-session system. Section 12 lists the limits.

Supabase plan. Dwarpal requires **Supabase, Free plan or above**. The Free plan is the floor; paid plans work unchanged.

2. Design decisions

Topic	Decision
Module shape	No auth service. SQL migrations in Supabase Postgres, a static browser kit and a Node library with a CLI. Your application data stays in your own store, whatever database that is, behind your Node endpoint, which calls the library in-process per request.
Delivery	One versioned npm package, <code>@briqvent/dwarpal</code> . Consumers pin a version.
Roles	Client-defined rows, not a closed set. Every role carries three reserved flags the kit enforces: <code>self_assignable</code> , <code>manages_members</code> , <code>mfa_required</code> . A role is never both <code>self_assignable</code> and <code>manages_members</code> .
Permissions	Role-based access control with client-defined permission keys and a role-to-permission map, evaluated in-process from a live snapshot per request. Type-level only; row ownership stays with the consumer.
Several roles	A user may hold several roles for one client. Effective permissions are the union over active roles (section 4.4).
Permission model	A JSON file in the client's repository, applied by the operator with <code>auth-kit apply-model</code> in one transaction with a diff record. Editing from the SQL editor is a recovery path that must be exported back into the file. No admin UI and no model sync from the web application.

Topic	Decision
Who changes what	The operator owns the model and manager promotion. Managers grant and revoke non-manager roles. Neither a manager nor a model change can promote existing holders to a manager role.
Enrollment	Public enrollment happens once per user per client and is recorded durably with its initial grants. Later sign-ins never re-grant; a manager's revoke is durable.
Access paths	Two paths with different revocation guarantees: the Node path with one-request revocation, and the direct RLS path with revocation bounded by access-token expiry (section 5.14).
Session check	The Node path checks live on every request: signature, GET /auth/v1/user and a fresh access read. No cached mode.
MFA	A per-role flag. Permissions from an mfa_required role count only in an aal2 session. Clients that never set the flag never show MFA screens.
Browser tokens	localStorage via supabase-js on the static host, with the residual XSS risk stated (section 5.9). No cookie or backend-for-frontend adapter.
Tenancy	One Supabase project per website by default. Several clients in one project are supported but explicit.
Consumer links	Links between identities and your business records are stored and audited by your application. The kit provides no hook and no adapter for them.
Build	The browser bundle is built with esbuild inside the package; consumers never need esbuild.
Test environments	A local emulator of the used Supabase API subset serves as a development fixture; acceptance runs against a hosted Supabase project, including the SQL actor matrix.

Dependencies. Runtime: `@supabase/supabase-js` (MIT) and `jose` (MIT). Development: `esbuild`. Considered and not adopted: archived or deprecated Supabase UI and helper packages, `@supabase/ssr`, full auth frameworks (`better-auth`, `next-auth`, `lucia`, `Keycloak`, `Ory Kratos`) and policy engines (`@casl/ability`, `node-casbin`). Evaluation here is set membership over a per-request snapshot, which needs no engine.

3. Architecture

packages/core/ redirect rules, no I/O. evaluation fixture guards against	pure JS: contract types, config and model schemas and validators, permission evaluation (<code>can</code> , <code>canAll</code> , <code>canAny</code> , <code>explain</code>), error taxonomy; Test entry point <code>@briqvent/dwarpal/testing</code> exports the shared (principals per role, MFA state and client) so consumers test their the same principals the kit tests with.
packages/browser/ screens;	headless controller and state machine on <code>supabase-js</code> ; optional default theme, copy and routes from configuration.
packages/server/ requireRole, requireMfa;	Node 22+: <code>createAuthServer</code> , <code>resolveSession</code> , <code>requirePermission</code> , manager operations; operator operations; CLI. Optional export <code>@briqvent/dwarpal/server/hono</code> (generic middleware; Hono is an optional peer dependency).
packages/emulator/ RPC endpoints.	development-only fake of the Supabase Auth API subset and the <code>auth_kit</code>
supabase/migrations/ profiles, public_clients view)	numbered SQL: schema <code>auth_kit</code> (exposed: wrappers, RLS helpers, and schema <code>auth_kit_private</code> (tables and implementations); grants

```

asserted.
examples/protected-consumer/ minimal Node server proving the contract per role and
permission, with its own
                               SQLite link table and the own/any guard.
examples/rls-consumer/       one consumer table in Supabase Postgres with a policy using the
RLS helpers.
examples/example-studio/     a second, synthetic brand with different role keys, route
prefix and build tool (Vite).
docs/                        this design, the low-level design and the user manual.

```

The package contains no consumer-specific code: no application adapters, no brand names, no role names and no permission keys outside the examples.

Runtime shape on a static-site host: static pages under the configured route prefix, served by the host's web server; one bundled script; the Content Security Policy's `connect-src` extended with the Supabase project origin only. Node consumers call `packages/server` in-process per request. The kit has no network API of its own. Its three surfaces are:

Surface	Caller	Credential	Can do
Browser	the end user's browser	publishable key and that user's access token	Supabase Auth endpoints; two SQL wrappers that act only on the caller's own user
Server library	your web application, in-process	publishable key and the request's own access token	resolve a session, evaluate permissions, forward manager actions under the manager's own token
Operator	a person or deploy step with the project's secret key	secret key and a Supabase Management API token	migrations, client registration, the permission model, the first manager, MFA reset, diagnostics

4. Contract 0.5

`AUTH_CONTRACT_VERSION = '0.5'` is exported from `packages/core`. Consumers assert it at startup and refuse to start on a mismatch.

Compatibility rule: within 0.x, additive optional fields may be added without a version change. Removing a field or changing an error code changes the contract version and is announced in the release notes.

4.1 Types and the Principal

```

type RoleKey = string;           // client-defined, e.g. 'supervisor' | 'agent' | 'buyer'
type PermissionKey = string;     // client-defined; recommended 'resource:action' or
'resource:action:scope'
type Provider = 'email' | 'google';
type Aal = 'aal1' | 'aal2';

interface RoleFlags { selfAssignable: boolean; managesMembers: boolean; mfaRequired: boolean; }
interface Identity { userId: string; verifiedEmail: string | null; providers: Provider[]; }
interface Membership {
  clientId: string; roleKey: RoleKey; flags: RoleFlags;
  grantedAt: string; grantedVia: 'join' | 'manager' | 'operator';
}
interface Access {

```

```
    clientId: string;                // the server's configured client
    enrolledAt: string | null;       // when public enrollment happened for this client; null if
never
    roles: RoleKey[];                // all roles held for that client, active or not (display
only)
    activeRoles: RoleKey[];          // roles whose MFA requirement the session satisfies
    permissions: PermissionKey[];    // union over activeRoles
    mfaPending: boolean;             // a held role is withheld for lack of aal2
}
interface Principal {
    identity: Identity;
    memberships: Membership[];       // for the configured client only
    access: Access;                   // authorization decisions use this only
    session: { id: string; aal: Aal; issuedAt: string; expiresAt: string; checkedAt: string };
}
interface Profile {
    userId: string; displayName: string | null; contactEmail: string | null; contactPhone:
string | null;
}
```

`memberships` and `access` are two views of the same single `effective_access(clientId)` read. **Client scope:** a user who manages client A and is a buyer of client B, resolved by a server configured for B, yields a principal that mentions B only. Nothing is read for other clients.

4.2 Roles and reserved flags

Roles are rows the operator defines per client. Their keys, descriptions and permission sets are client data. Three flags have meaning the kit enforces:

Flag	Meaning	Set by
<code>self_assignable</code>	<code>join_client</code> grants the role to a verified user at their first and only enrollment	operator, in the model
<code>manages_members</code>	a holder may grant and revoke non-manager roles for this client; the first manager must hold such a role	operator, in the model
<code>mfa_required</code>	the role's permissions count only in an <code>aal2</code> session; the browser kit prompts enrolment	operator, in the model

A role can never be both `self_assignable` and `manages_members`, so public sign-up can never yield management authority. A client with no `self_assignable` role is invite-only.

4.3 Permission keys: own, self and any

Because permissions are a union over roles, a lesser role can hand a user a key. A key must therefore mean exactly one thing. The convention:

Suffix	Meaning	Guard predicate your code adds
<code>:any</code>	all rows of the resource	none
<code>:own</code>	rows the principal owns; <code>own</code> always refers to the row being acted on	the row's owner equals <code>principal.identity.userId</code> (or your audited link row says so)
<code>:self</code>	actions whose narrow scope is the principal as the target of the action, for example "assign to myself"	the target user id equals <code>principal.identity.userId</code>

`own` never refers to the target of an action; that is what `:self` is for. A key without a scope, such as `members:manage`, is fine when there is no `own`/any distinction.

The consumer guard checks the broad key first, then the narrow key, then ownership:

```
if (!can(principal, 'orders:read:any')) {           // broad branch needs an ACTIVE role that grants it
  requirePermission(principal, 'orders:read:own'); // throws forbidden or mfa_required
  if (order.userId !== principal.identity.userId) throw new AuthError('forbidden');
}
```

A user holding `buyer` and the MFA-required `agent` role at `aal1` fails the first line (the `agent` role is inactive), passes the second, and is stopped by the ownership check on another buyer's order. `explain(principal, 'orders:read:any').withheld` tells the page to offer MFA.

4.4 Evaluation semantics

- A role is **active** when `mfa_required` is false or `session.aal = 'aal2'`.
- `access.permissions` is the union of permission keys over active roles. A permission is granted if any active role grants it.
- `mfaPending` is true when at least one held role is inactive.
- Unknown keys are not granted (fail closed), and `explain` reports no grantor.
- `access.roles` lists held roles for display. **No authorization helper reads it.** `requireRole` and the SQL `has_role` helper test active roles only.
- The same rule is implemented in SQL (`effective_access` and the RLS helper implementations) and in `packages/core`. Both are tested against one fixture set.

4.5 Server library

- `createAuthServer({ supabaseUrl, publishableKey, clientId, clockToleranceSeconds: 5, fetch? })`. `clientId` is fixed at construction and never read from a request. No secret key is needed to resolve sessions. Any Auth, JWKS, PostgREST or network failure raises `AuthError('unavailable')`, never an empty or stale principal.
- `resolveSession(request) → Principal | null`:
 1. `jose` verifies the signature via JWKS, `iss`, `aud`, and `exp/nbf/iat` with 5 s tolerance; requires `is_anonymous = false`; reads `session_id` and `aal`.
 2. `GET /auth/v1/user` with the token. A signed-out, banned or deleted user yields `invalid_token`.
 3. `effective_access(clientId)`, called as the user, returns enrollment, roles, flags and permissions in one round trip.

It returns `null` when the request carries no bearer token. Maximum revocation lag: one request. The library holds no session state and no cache other than JWKS public keys.

- `can(principal, key)`, `canAll(principal, keys)` and `canAny(principal, keys)` are pure functions over `access.permissions`.
- `explain(principal, key) → { allowed, via: RoleKey[], withheld: { role, reason: 'mfa_required' }[] }`.
- `requirePermission(principal, key)` returns or throws `forbidden`; it throws `mfa_required` instead when the key would be granted by a withheld role.

- `requireRole(principal, roles)` tests `access.activeRoles`. `requireMfa(principal)` requires `aal2`.
- **Manager operations** run under the manager's own token, with no secret key: `grantMembership(userId, roleKey, requestId)` and `revokeMembership(userId, roleKey, requestId)`. SQL rechecks every rule (section 4.6).
- **Operator operations** read the secret key from the environment only, use a separate `supabase-js` instance and are never given a user token: `registerClient`, `applyModel(model, { dryRun })`, `exportModel(clientId)`, `bootstrapManager({ userId } | { email }, roleKey, requestId)`, `revokeManager`, `mfaReset(userId, requestId)`, `checkConfig`, `migrate`, `doctor({ probe? })`. The CLI is a thin wrapper over these.
- `honoMiddleware(authServer)`, exported from `@briqvent/dwarpal/server/hono`, sets `c.var.principal` or responds 401 or 503. It contains no consumer or client knowledge.

No consumer adapters. The kit knows nothing about your leads, orders or customers. Build whatever actor your code needs from `Principal` (`identity.userId`, `memberships`, `access.permissions`, `access.activeRoles`) inside your own package, and store and audit any identity-to-record links yourself.

Bootstrap by email. The canonical input of `bootstrapManager` is `userId`, verified with `auth.admin.getUserById` (the user must exist with a confirmed email). `email` is a convenience: an `auth.admin.listUsers` scan (page size 1000, at most 10 pages) that must reach the end of the listing before any result is accepted. It then requires exactly one confirmed match, refusing with `unknown_user` on zero and `ambiguous_user` on several. If the page cap is reached while more users exist, the CLI refuses with `lookup_incomplete`, writes nothing and tells the operator to use `--user-id`; an incomplete scan never claims uniqueness. `auth.admin.inviteUserByEmail` is used only with `--invite`, and the CLI prints the returned id for the rerun.

Diagnostics. `doctor` by default inspects the catalog with the secret key: `has_schema_privilege`, `has_table_privilege` and `has_function_privilege` for `anon`, `authenticated`, `PUBLIC` and `service_role` against the grant table in section 4.6, plus schema version, exposed schemas, asymmetric signing keys, the redirect allow-list, memberships without events and the model hash against the file. `doctor --probe` also signs in a disposable user the operator names (`--probe-email`, credentials from the environment) with the publishable key and makes real calls as `authenticated` and as `anon`. The report says which mode produced each line.

4.6 SQL surface

Two schemas:

- `auth_kit` is exposed through PostgREST and holds only `security invoker` wrappers, the RLS helpers, the profiles table and the `public_clients` view.
- `auth_kit_private` is **never** in the project's exposed schemas. It holds all other tables and all `security definer` implementations, each with `set search_path = ''`.

Wrappers validate argument shapes and call the private implementation. Implementations read the actor from `auth.uid()`, `auth.role()` and `auth.jwt()` -> `'aal'`, never from arguments.

Tables

In `auth_kit_private`:

- `clients(client_id PK, display_name, signup_policy open|closed, state registered|live, created_at)`. `state` becomes `live` on the first successful `bootstrap_manager` and never returns to

registered.

- `roles(client_id, role_key, description, self_assignable, manages_members, mfa_required, PK(client_id, role_key), CHECK not (self_assignable and manages_members))`.
- `permissions(client_id, permission_key, description, PK(client_id, permission_key))`.
- `role_permissions(client_id, role_key, permission_key)` with composite foreign keys to `roles` and `permissions`, on delete restrict.
- `memberships(user_id, client_id, role_key, granted_at, granted_by, granted_via join|manager|operator, PK(user_id, client_id, role_key))`, foreign key to `roles` on delete restrict.
- `enrollments(user_id, client_id, enrolled_at, granted_roles text[], PK(user_id, client_id))`. One row per public enrollment. No function ever updates or deletes it. It is the durable record that the initial grant happened.
- `membership_events(id, request_id unique nullable, payload_hash, result, action join|grant|revoke|bootstrap|revoke_manager|mfa_reset, user_id, client_id, role_key, actor_user_id, actor_kind user|operator, at)`. `client_id` is null only for `mfa_reset`, whose scope is the whole project.
- `model_events(id, request_id unique, model_hash, client_id, diff jsonb, actor_kind operator, at)`, written for applied changes only.
- `request_log(request_id uuid PK, client_id nullable, operation, actor_id text, payload_hash, state pending|completed, result jsonb, factors_seen uuid[] nullable, run_token uuid nullable, started_at, at)`. See the request semantics below.

In `auth_kit`: `profiles(user_id PK, display_name, contact_email, contact_phone, updated_at)`.

Grants

Each migration ends with an assertion query that checks every row of this table; `doctor` reruns the same query.

Object	anon	authenticated	service_role	PUBLIC
schema <code>auth_kit</code> USAGE	yes (needed to resolve the RLS helpers from a consumer policy; nothing else is executable)	yes	yes	revoked
schema <code>auth_kit_private</code> USAGE	no	yes (to execute granted implementations; not API-exposed)	yes	revoked
all <code>auth_kit_private</code> tables	none	none (reads only through <code>effective_access</code> and the helper implementations)	all (bypasses RLS)	none
view <code>auth_kit.public_clients</code>	none	<code>SELECT client_id, display_name</code>	all	none
table <code>auth_kit.profiles</code>	none	<code>SELECT, INSERT, UPDATE</code> of the caller's own row (RLS and column grants)	all	none

Object	anon	authenticated	service_role	PUBLIC
default privileges on functions and tables in both schemas	revoked	revoked; each grant explicit	explicit	execute revoked through alter default privileges, and again with revoke execute on all functions in schema ... from public at the end of every migration
user wrappers ensure_profile, join_client, effective_access, grant_membership, revoke_membership	revoked	EXECUTE	EXECUTE	revoked
operator wrappers register_client, apply_model, export_model, bootstrap_manager, revoke_manager, mfa_reset_begin, mfa_reset_note, mfa_reset_finish	revoked	revoked	EXECUTE	revoked
RLS helper wrappers has_permission(text, text), has_role(text, text), has_aal2()	EXECUTE (return false when auth.uid() is null, before touching the private schema)	EXECUTE	EXECUTE	revoked
auth_kit_private implementations	revoked	EXECUTE only on the five user-callable implementations and the helper implementations	EXECUTE	revoked

API exposure versus SQL privilege. These are two separate fences. PostgREST routes only to exposed schemas, and `auth_kit_private` is never exposed, so no HTTP request can reach an implementation whatever its EXECUTE grant. The SQL privileges protect the implementations from a caller who can already run SQL as `authenticated`, for example through a definer function in a consumer schema. Both are asserted, and they are tested differently: routing over HTTP, privileges by role and claim impersonation in SQL.

Locking

The first statement of every write implementation is `perform pg_advisory_xact_lock(hashtext('auth_kit: ' || client_id))`. All writes for one client, model and membership alike, therefore serialise on one lock. Authority is checked **after** the lock, so a manager revoked or a model changed by a concurrent transaction is seen. Reads (`effective_access`, the helpers) take no lock. The three `mfa_reset_*` functions take the project-wide lock `hashtext('auth_kit:mfa_reset')` instead, because the operation has no client.

Request semantics and `request_log`

Commands that carry a `request_id` form a closed list: `grant_membership`, `revoke_membership`, `bootstrap_manager`, `revoke_manager`, `apply_model` (when `dry_run` is false) and the three `mfa_reset_*` functions.

- Each looks up `request_log` by id after taking the lock.
 - Found with the same `payload_hash`: return the stored `result` and write nothing.
 - Found with a different hash: raise `request_conflict` and write nothing.
 - Not found: run the command, then insert a `request_log` row with the outcome, **even when nothing was mutated** (unchanged model, `already_member`, `not_member`), plus an event row only if something changed.
- There is no third outcome: every retry is either the stored result or `request_conflict`. This holds for commands whose first run changed nothing, because the no-op outcome is recorded too.
- The fingerprint is `sha256` of the canonical JSON of `{operation, client_id, actor_id, payload}`. `payload` is the function's arguments without `request_id`; for `apply_model` it is the canonical model hash. `actor_id` is `auth.uid()::text` for manager calls and the literal `operator` for service-role calls. Because the actor is part of the fingerprint, one manager cannot replay another manager's id.
- Event tables record **mutations**; `request_log` records **outcomes**. They are joined by `request_id`.
- `state`, `factors_seen`, `run_token` and `started_at` serve `mfa_reset` only. Every other command inserts a `completed` row in the same transaction as its work. `client_id` is null only for `mfa_reset`.

Commands without a `request_id`, and why: `join_client` is idempotent on the `enrollments` primary key; `register_client` is idempotent on `client_id` (a repeat with a different display name or policy is an update, not a conflict); `ensure_profile` is an upsert; `apply_model` with `dry_run` writes nothing and ignores any id; `export_model` and `effective_access` are reads.

House pattern. The same rule is the recommended shape for your own idempotent commands: a fresh UUID per logical action, reused on retry; a fingerprint of operation, client, actor and canonical payload stored with the first result; the stored result on an identical retry and a conflict otherwise; and a row written even when the first run changed nothing.

Functions

Function	Caller	Rule
<code>ensure_profile()</code>	authenticated	Upserts the caller's own profile row; idempotent.
<code>join_client(client_id)</code>	authenticated	Lock. Reads <code>auth.users.email_confirmed_at</code> (the token is not trusted): <code>unconfirmed</code> → <code>email_unverified</code> . Then, in order: client missing → <code>unknown_client</code> ; <code>enrollments</code> row exists → <code>already_enrolled</code> , nothing written or re-granted , whatever the current memberships are; <code>signup_policy = closed</code> → <code>closed</code> ; no <code>self_assignable</code> role → <code>no_default_role</code> , nothing written so the join stays retryable. Otherwise inserts the <code>enrollments</code> row with the granted role list, one membership per <code>self_assignable</code> role and one <code>join</code> event per membership, and returns <code>enrolled</code> . One transaction: a failure before commit leaves no enrollment row. Concurrent first joins serialise on the lock; the second returns <code>already_enrolled</code> .
<code>effective_access(client_id)</code>	authenticated	Read-only. Returns <code>enrolled_at</code> and the caller's memberships for that client with each role's flags and permission keys.

Function	Caller	Rule
<code>grant_membership(user_id, client_id, role_key, request_id)</code>	authenticated manager	Lock, then the <code>request_id</code> check, then manager authority: the caller must hold at least one <code>manages_members</code> role for that client that is active . Manager roles held but none active → <code>mfa_required</code> ; none held → <code>forbidden</code> . The target role exists and is not <code>manages_members</code> ; the target user exists and is confirmed; a manager cannot target themselves. Inserts, or returns <code>already_member</code> ; one event. This is the only path that restores a self-assignable role a manager revoked earlier.
<code>revoke_membership(user_id, client_id, role_key, request_id)</code>	authenticated manager	Same caller checks. The target role is not <code>manages_members</code> . Deletes, or returns <code>not_member</code> ; one event. Durable: <code>join_client</code> never restores the row.
<code>bootstrap_manager(user_id, client_id, role_key, request_id)</code>	service role	Lock and <code>request_id</code> check. The role has <code>manages_members</code> (otherwise <code>unknown_role</code> if it does not exist); the user is confirmed, otherwise <code>email_unverified</code> . Inserts, or returns <code>already_member</code> . Sets <code>clients.state = live</code> . Event with <code>actor_kind = operator</code> on insert; <code>request_log</code> row always.
<code>revoke_manager(user_id, client_id, role_key, request_id)</code>	service role	Lock and <code>request_id</code> check. Refuses with <code>last_manager</code> if no other membership in a <code>manages_members</code> role would remain for the client. <code>not_member</code> when absent. <code>request_log</code> row always.
<code>register_client(client_id, display_name, signup_policy)</code>	service role	Idempotent on <code>client_id</code> ; <code>state = registered</code> .
<code>apply_model(client_id, model jsonb, request_id, dry_run)</code>	service role	Lock and <code>request_id</code> check (payload hash = model hash). Validates the whole model: at least one <code>manages_members</code> role; no <code>self_assignable</code> manager; every mapped key declared; no deletion of a role that users hold; no deletion of a permission that is still mapped unless it is unmapped in the same model; no <code>manages_members := true on a role that has holders</code> (refused with the holder list); and, if the client is live, at least one existing membership in a <code>manages_members</code> role must remain (refused <code>no_manager_would_remain</code>). Computes the diff and annotates each change with its reach: <code>future_joiners</code> for <code>self_assignable</code> changes, <code>holders: N</code> for permission mapping changes. Applies in one transaction with one <code>model_events</code> row. <code>dry_run</code> returns the annotated diff only. Reapplying the same model gives an empty diff and no <code>model_events</code> row, but a <code>request_log</code> row with <code>result = unchanged</code> .
<code>export_model(client_id)</code>	service role	Read-only. Returns the current model as the canonical JSON of the file format, plus the last applied hash.
<code>mfa_reset_begin(user_id, request_id)</code>	service role	See section 4.7.
<code>mfa_reset_note(request_id, run_token, factor_ids)</code>	service role	See section 4.7.
<code>mfa_reset_finish(request_id, run_token, factors_deleted)</code>	service role	See section 4.7.

RLS helpers

For consumer tables that live in Supabase Postgres:

Object	Definition
<code>auth_kit.has_permission(client_id text, permission_key text)</code> returns boolean	language sql stable security invoker; select case when auth.uid() is null then false else auth_kit_private.has_permission_impl(\$1, \$2) end.
<code>auth_kit.has_role(client_id text, role_key text)</code> returns boolean	Same shape. True only for an active role: held, and mfa_required satisfied by auth.jwt() ->> 'aal'.
<code>auth_kit.has_aal2()</code> returns boolean	select coalesce(auth.jwt() ->> 'aal', 'aal1') = 'aal2'; no private call.
<code>auth_kit_private.has_permission_impl, has_role_impl</code>	security definer, stable, set search_path = '', owned by the migration role. Read memberships, roles and role_permissions for auth.uid() and the argument client, apply the same activation rule as effective_access, and return false for an unknown client, role or key. Never raise for an absent row.

A consumer policy then reads, for example:

```
using (
  auth_kit.has_permission('acme', 'orders:read:any')
  or (auth_kit.has_permission('acme', 'orders:read:own') and owner_id = auth.uid())
)
```

4.7 MFA reset: reserve, then act, under one claim

`mfaReset(userId, requestId)` crosses Supabase Auth and SQL, so it cannot run in one transaction. Its rule is **reserve, then act, under one claim**. The id is bound in SQL before the first Auth call; a replay, a changed target or a concurrent run is stopped before any deletion; and exactly one runner holds the right to complete the row at any time.

1. **Reserve.** `mfa_reset_begin(user_id, request_id)` runs first, before any Auth call, under the project-wide lock. It looks up `request_log` by id and answers one of:

Outcome	When	Effect
proceed	no row	Inserts a pending row with a fresh <code>run_token</code> and <code>started_at = now()</code> ; returns the token.
stored result	same fingerprint, completed	Returns the stored result; the CLI prints it and makes no Auth call.
resume	same fingerprint, pending, <code>started_at</code> older than the 120 s lease	Claim: the same statement replaces the row's <code>run_token</code> with a fresh one and sets <code>started_at = now()</code> ; returns the new token and the recorded factor list.
request_in_progress	same fingerprint, pending, inside the lease	Raised; another run holds the claim, including a takeover made a moment ago.
request_conflict	different fingerprint, i.e. another user behind the same id	Raised.

The last two are raised before Auth is touched.

2. **List.** `auth.admin.mfa.listFactors`.
3. **Note.** `mfa_reset_note(request_id, run_token, factor_ids)` records the verified TOTP factor list on the pending row once. It is a no-op when the row already holds a list, so a resumed run keeps the original list.

4. **Delete.** `deleteFactor` for each factor in the recorded list. "Factor not found" counts as already deleted.
5. **Finish.** `mfa_reset_finish(request_id, run_token, factors_deleted)` sets `state = completed`, stores `result = reset` (or `no_factors` when the recorded list was empty) with both lists, keeps the completing token on the row, and writes the one `membership_events` row with `action = mfa_reset`, `client_id` null and `actor_kind = operator`.

Claim check. `note` and `finish` compare the caller's `run_token` with the row's **before any state check**:

Function	Row state	Result
<code>note</code> or <code>finish</code>	absent	<code>request_conflict</code>
<code>note</code> or <code>finish</code>	token differs, <code>pending</code> or <code>completed</code>	<code>run_superseded</code> ; nothing written
<code>note</code>	<code>completed</code> , caller's own token	<code>request_conflict</code> (a <code>note</code> after <code>finish</code> is a runner bug)
<code>note</code>	<code>pending</code> , caller's own token	sets <code>factors_seen</code> only if null; returns the recorded list
<code>finish</code>	<code>completed</code> , caller's own token	the stored result (the claim holder's own retry)
<code>finish</code>	<code>pending</code> , caller's own token	completes the row and writes the event

So only the current claim holder can record the list or complete the row, and exactly one event exists per request id. A superseded runner that calls `finish` after the takeover completed gets `run_superseded`, not the stored result, so it never believes it owned the run.

Deadline. Every admin API call has a 20 s timeout. Before each one the CLI checks that its claim is younger than 100 s (the lease minus that timeout), measured on its own monotonic clock from the moment it sent `begin`. That moment is never later than the row's `started_at`, so no clock is compared across machines. Past the deadline the runner stops with `lease_expired`, makes no further Auth call and tells the operator to rerun with the same id; the row stays `pending` with its list. A live runner therefore never makes an Auth call after its lease could have passed to another runner, and a slow runner gives up instead of racing. There is no lease renewal and no job queue. Normal runs finish in seconds. A pathological run against a user with many factors and a slow API may need more than one rerun, each of which resumes with the original list.

If a superseded runner nevertheless reaches Auth (a call that overran its timeout), it can only repeat deletions of factors in the recorded list, which are idempotent, and its `note` and `finish` are refused, so the audit stays with the claim holder.

Fingerprint and scope. The fingerprint is `{operation: 'mfa_reset', client_id: null, actor_id: 'operator', payload: {user_id}}`. Neither the factor list nor the token is part of it, so a retry after success returns the stored result without an Auth call. The scope is the whole project because a factor reset affects the user across every client. A crash after some deletions resumes with the original list intact; the audit never claims the factors were absent. The experimental recovery-code API is not used in 0.x. A user at `aal2` may also remove their own factor through the browser kit.

4.8 Error sets

Library errors (closed set, `AuthError.code`): `no_token`, `invalid_token`, `expired`, `email_unverified`, `mfa_required`, `forbidden`, `unavailable`, `provider_unavailable`, `config_invalid`, `model_invalid`, `request_conflict`.

CLI-only outcomes. For `bootstrap-manager`: `unknown_user`, `ambiguous_user`, `lookup_incomplete`, `setup_pending` (invitation sent). For `mfa-reset`:

- `request_in_progress`: a run with the same id is inside its lease; retry after it ends.

- `run_superseded`: this run's claim was taken over after its lease expired; its `note` or `finish` wrote nothing; rerun with the same id to see the outcome.
- `lease_expired`: this run stopped itself before an admin API call because its claim could have expired; the row is still pending; rerun with the same id.

SQL function results (returned, not thrown, unless noted): `enrolled`, `already_enrolled`, `unknown_client`, `closed`, `no_default_role`, `already_member`, `not_member`, `unchanged`, `reset`, `no_factors`. Refusals: `unknown_role`, `last_manager`, `no_manager_would_remain`, and holder-list refusals from `apply_model`.

5. Trust boundaries and security decisions

1. **Roles and permissions are server-authoritative** through section 4.6. `raw_user_meta_data` and `app_metadata` are never read for authorization; only table rows grant anything.
2. **Public sign-up and join yield only self assignable roles**, after email verification, through `join_client`, **once per user per client**. A client with no such role is invite-only; a closed client returns `closed`.
3. **Manager bootstrap is deliberate and operator-only**: `auth-kit bootstrap-manager --client <id> --role <role> --user-id <uuid>`, run with the secret key from the environment. It refuses unless the user exists with a verified email, is idempotent and is audited. Managers cannot create managers, and neither can a model change.
4. **Verified identity is not a supplied contact email**. `verifiedEmail` comes only from the Auth user record. `Profile.contactEmail` is display text and never a lookup key.
5. **Linking rules**. A historical record becomes reachable by a user only through an explicit link row that your application stores, with the exact record id, the exact `userId`, the acting staff identity, the time and the reason, and with an unlink path. A link is never created from an email match, verified or not. Supabase's automatic identity linking decides which Auth identities belong to one `userId`; it never touches your links.
6. **Account linking policy**. Supabase's documented behaviour applies: automatic linking on verified email only, unconfirmed identities removed on link, and a neutral response for password sign-up over an existing email. Manual identity linking is disabled in 0.x.
7. **Redirects**. The Supabase allow-list uses exact callback URLs per environment, with no wildcards. The kit validates `next` by parsing it with `new URL(next, origin)` and rejecting it if the origin differs, if the raw value starts with `//` or contains `\`, `%2f`, `%5c`, control characters, whitespace or a fragment. It then normalises the path and requires an exact match against the client's `allowedReturnPaths`; otherwise it falls back to the client's default. The validated `next` is stored in `sessionStorage` under the PKCE flow id and read back only on that flow's callback. After `exchangeCodeForSession` or `verifyOtp`, `history.replaceState` strips `code`, `token_hash` and `type`.
8. **Secrets**. The browser gets only the project URL and the publishable key. The secret key and provider secrets live in the Supabase dashboard and the operator's local environment. Log lines are built from allow-listed fields. The model file contains no secrets by construction and is safe in the client's repository.
9. **Sessions in the browser**. PKCE is explicit. Tokens are stored in `localStorage` on the static host. Residual risk: a script injected into the site's origin can read tokens. Mitigations are a strict CSP, no third-party or inline scripts, 30-minute access tokens, live server checks and global sign-out. Sign-out uses `scope: 'global'` and clears local state even if the network call fails. On the server, the JWKS is cached per Supabase's guidance, and `checkConfig` fails when the project has no asymmetric signing key.
10. **MFA**. Enrolment is prompted when the user holds any `mfa_required` role. Permissions from such roles are withheld until `aal2`; other roles keep working. After verification the browser kit refreshes the session so the new `aal` reaches the server. A lost authenticator is handled by `auth-kit mfa-reset` (section 4.7).
11. **Browser `getPrincipal()` is display state only**. Every authorization decision happens on your server through `resolveSession`, or in SQL through the helpers.

12. **Where sessions live.** Supabase Auth is the session authority: it issues access tokens (30 minutes recommended) and rotating refresh tokens, stores sessions, and handles sign-out, global sign-out and MFA level. The server library is **stateless**: no session store, no membership or permission cache, one live check per request. Its only in-memory state is the JWKS public-key cache. Horizontal scaling needs no session synchronisation. The cost is one Auth call and one PostgREST call per authenticated request per instance. This paragraph describes the Node path; see item 14 for the direct path.
13. **Platform limits.** (a) The data is hosted in Supabase's cloud, so "same machine" applies to your web application and the library, not to the database; every hop is TLS with a per-user token or the operator key. (b) The secret key is root for the project: a holder who writes tables directly instead of calling the functions can break the invariants. The kit keeps the key off the web application and the browser, records every sanctioned write as an event, and `doctor` reports memberships without events and model-hash drift.
14. **Two access paths, two revocation guarantees.** RBAC answers "may a buyer read orders", never "may this buyer read order 123"; ownership is your design.
 - **Node path (the supported path for sensitive records).** Browser → your Node endpoint → `resolveSession` (signature, live Auth check, fresh access read) → `requirePermission` → your ownership rule → your data. Sign-out, ban, deletion, membership revoke and model change all take effect on the next request.
 - **Direct RLS path (supported, with a weaker guarantee).** Browser → PostgREST with the user's token → your table policy using `auth_kit.has_permission()`. PostgREST validates the token's signature and expiry and evaluates the policy. **It never calls** `GET /auth/v1/user`, so a signed-out, banned or deleted user keeps access until the access token expires (at most the configured lifetime). Membership revoke and model changes are immediate on this path too, because the helpers read the tables live. The kit calls this "revocation bounded by JWT expiry" and never claims one-request revocation for it. Put only public or low-risk data behind it.
15. **Authority split.** The **operator** is the trusted administrator of a deployment: whoever holds that project's secret key and runs the CLI. The operator owns the model and manager promotion. Client managers own people. A client's business owner is a manager, never the model author. Every adopter operates its own Supabase project, model and manager bootstrap.
16. **Concurrency and retries.** All writes for one client serialise on one advisory transaction lock, with authority checked after the lock; reads take no lock. Every retry with a `request_id` is either the stored result or `request_conflict` (section 4.6). `mfa_reset` binds its id before the first Auth call and admits one claim holder at a time (section 4.7). Write throughput per client is bounded by this serialisation, which suits small and medium sites.

6. Configuration

Per client, validated by `packages/core`:

```
interface ClientConfig {
  clientId: string; // must match auth_kit_private.clients.client_id
  supabaseUrl: string;
  publishableKey: string;
  routes: { prefix: '/account', signIn, signUp, verify, callback, forgot, reset, mfa, signOut }; // all overridable
  origin: string;
  allowedReturnPaths: string[];
  defaultReturnPath: string;
  providers: { email: boolean; google: boolean };
  session: { accessTokenMinutes: 30 }; // documented; the value itself is set in the
```

```
Supabase dashboard
brand: { name, logoUrl, colors, fontStack };
copy: Partial<Record<CopyKey, string>>;
selfSignup: boolean;           // must agree with clients.signup_policy; checkConfig
verifies
}
```

6.1 The model file

`auth-model.json` is validated by the same package and applied by the operator. The example below is for a fictional supply shop, client id `acme`, whose website uses an orders service and a leads service. The permission keys belong to those example services; the kit neither knows nor validates them beyond "declared and mapped".

```
{
  "client": "acme",
  "roles": {
    "supervisor": {
      "manages_members": true, "mfa_required": true,
      "permissions": ["members:manage",
        "orders:read:any", "orders:create:any", "orders:update:any",
"orders:cancel:any",
        "orders:reopen:any", "orders:grant:any",
        "documents:upload:any", "documents:download:any",
"documents:review:any",
        "leads:read:any", "leads:create:any", "leads:update:any",
"leads:assign:self",
        "leads:assign:any", "leads:convert:any", "contacts:update:any"]
    },
    "agent": {
      "mfa_required": true,
      "permissions": ["orders:read:any", "orders:create:any", "orders:update:any",
"orders:cancel:any",
        "orders:grant:any",
        "documents:upload:any", "documents:download:any",
"documents:review:any",
        "leads:read:any", "leads:create:any", "leads:update:any",
"leads:assign:self",
        "leads:convert:any"]
    },
    "buyer": {
      "self_assignable": true,
      "permissions": ["orders:read:own", "documents:upload:own", "documents:download:own"]
    }
  },
  "permissions": {
    "members:manage": "grant and revoke non-manager roles",
    "orders:read:own": "read orders the user is linked to; the link is the orders service's
audited row",
    "orders:read:any": "read any order",
    "orders:create:any": "create an order",
```

```

"orders:update:any": "ordinary workflow actions, notes, milestones",
"orders:cancel:any": "cancel an order",
"orders:reopen:any": "reopen a closed order",
"orders:grant:any": "link or unlink a buyer to an order",
"documents:upload:own": "upload to an order the user is linked to",
"documents:upload:any": "upload on a buyer's behalf",
"documents:download:own": "",
"documents:download:any": "",
"documents:review:any": "add or waive requirements, review versions",
"leads:read:any": "list, detail, search",
"leads:create:any": "create a lead manually",
"leads:update:any": "stage changes, notes",
"leads:assign:self": "assign a lead to oneself: the target assignee must equal the
principal's user id",
"leads:assign:any": "assign a lead to any member",
"leads:convert:any": "convert a lead to an order",
"contacts:update:any": "edit a contact's personal details"
}
}

```

In this example `supervisor` and `agent` are `mfa_required`, so at aall an agent has no active `:any` key and the page shows the MFA prompt, while `buyer` keeps working. Which roles hold which keys is the client's decision.

Validation rejects a role that is both `self_assignable` and `manages_members`, a mapped but undeclared permission, a model with no `manages_members` role, and the live-client and holder rules of `apply_model` in section 4.6.

Adding `self_assignable` to a role later, or adding a new self-assignable role, reaches **future joiners only**, and the dry-run diff says so. Existing enrolled users receive it through a manager grant.

The `examples/example-studio/` brand uses different role keys (`owner`, `editor`, `member`), a different route prefix and a different build tool, to show that the core carries no example-specific names.

7. Flows and screen states

Screen states: `idle`, `submitting`, `sent`, `error(code)`, `expired_link`, `already_used`, `mfa_enrol`, `mfa_challenge`, `setup_pending`, `no_access`, `signed_in`, `offline`.

- **Sign up** → confirmation email (template uses `{{ .TokenHash }}` and the configured verify route) → `verify0tp({ token_hash, type: 'email' })` on a button click, not on page load → signed in → `ensure_profile` → `effective_access` → if `enrolledAt` is null, `join_client` → `effective_access` again → `next`.
- `join_client` **outcomes in the browser**. `enrolled` or `already_enrolled` → continue. `unknown_client` or `no_default_role` → `setup_pending`: the operator has not finished setup; show a retry button, with no automatic loop. `closed` → `no_access`: a deliberately invite-only client; ask the site for access; no retry. `email_unverified` → the verification screen.
- **Sign in** → MFA challenge if enrolled → `ensure_profile` → `effective_access` → join only if not yet enrolled → if `mfaPending` and no factor is enrolled, `mfa_enrol` (skippable only if the client marks enrolment optional, in which case withheld roles stay withheld) → `next`. Unverified email →

`email_unverified` with resend and a countdown. A user whose membership was revoked is signed in with no membership and sees `no_access`.

- **Forgot password** → `resetPasswordForEmail` → configured reset route with `token_hash&type=recovery` → `verifyOtp({ type: 'recovery' })` → `updateUser({ password })`. An expired or reused link gives `expired_link`, never a silent sign-in. If `updateUser` fails, the kit keeps the recovery session on the reset screen with a retry, sets a `recovery_pending` marker, and every other kit page returns to the reset screen until the password is updated or the user signs out.
- **Google** → `signInWithOAuth` with the exact configured callback → `exchangeCodeForSession` → the same post-sign-in steps. A provider or exchange failure gives `provider_unavailable` with a fresh flow.
- **Sign out** → global revoke → public page.
- **Operator:** `migrate` → `doctor` → `register-client` → `apply-model` → `bootstrap-manager`. Zero-state outcomes are in the [low-level design](#).

8. Verification strategy

Layer	Target	Proves
Unit (node --test)	core, server	Config and model validation (self-assignable manager, undeclared key, held-role deletion, promotion by model, live-client manager loss); evaluation semantics against the shared fixture (union, MFA withholding, unknown key, own/any guard); redirect rules; JWT verification; fail-closed on injected failures; client scoping; the <code>testing</code> entry point exporting the same fixture consumers import; the Hono middleware; <code>request_conflict</code> including no-op first calls; <code>lookup_incomplete</code> on a capped user scan; MFA reset (every <code>mfa_reset_begin</code> outcome, resume with the recorded list, a single claim on takeover, <code>run_superseded</code> on a stale token, the CLI deadline with a fake clock, the admin API asserted untouched on replay and conflict); error redaction.
Integration	emulator	Browser flows, enrollment states (<code>enrolled</code> , <code>already_enrolled</code> , <code>closed</code> → <code>no_access</code> , <code>no_default_role</code> → <code>setup_pending</code>), MFA withholding, manager grant and revoke in the example consumer. A fixture, not proof.
Hosted acceptance	a Supabase project	Browser suite against the project; the SQL actor matrix as anon, enrolled member, non-manager staff, MFA-pending staff, manager of the same client, manager of another client and service role, across three model states (empty, the example model, a changed model), including one consumer RLS policy and a two-manager-role <code>aall</code> case; private implementations probed by SQL impersonation, never over HTTP; <code>apply_model</code> idempotency, refusals and concurrency; revoked session on both access paths ; the enrollment matrix; concurrent first joins; real SMTP, one interactive Google sign-in, TOTP enrol and challenge; <code>doctor</code> in catalog and probe modes.
Negative and security	both	Forged metadata and tokens, cross-client token, manager promoting a manager, model promoting holders, last-manager removal by revoke and by model, HTTP RPC into <code>auth_kit_private</code> (unroutable for every actor), SQL EXECUTE on private implementations by impersonated role, direct table access, <code>next</code> abuse, secret markers absent from the bundle, an email match yielding no records, the same <code>request_id</code> with a different payload, and a buyer with an inactive staff role reading another buyer's order.
Reuse	example-studio	Different role keys, prefix and build tool pass the suite with the core unchanged.
Host integration	a static-site host	The install contract followed literally; existing routes, metadata and sitemap unchanged; account pages <code>noindex</code> .

Reports keep emulator results, hosted results and pending setup distinct.

9. Test plan

Each case names the required outcome and where it runs. "Hosted" means against a real Supabase project; "emulator" means the development fixture.

Id	Case	Required outcome	Test
L1	Sign-up for an existing email	Same neutral response; no new membership or profile.	emulator + hosted
L2	Verification or recovery link opened twice, or prefetched	Second use → <code>expired_link</code> ; no second session; verification runs on click, not on load.	emulator + hosted
L3	Two PKCE starts before completion	The callback exchanges only its own flow; a mismatch → <code>provider_unavailable</code> with a fresh start.	emulator
L4	<code>join_client</code> response lost, or PostgREST briefly down	<code>setup_pending</code> with retry; any number of retries → exactly one enrollment, one membership per self-assignable role and one event each.	emulator + hosted
L5	Retried <code>bootstrap_manager</code> or <code>grant_membership</code> after a lost response	Same <code>request_id</code> → the stored result; one membership, one event, one <code>request_log</code> row. See L30 for the full matrix.	unit + hosted
L6	Membership revoked mid-session	Next request on the Node path is denied.	unit + hosted
L7	Token at <code>exp</code> or <code>nbf</code> , future <code>iat</code> , key rotation	5 s tolerance; rotated key accepted after a JWKS refresh; unknown <code>kid</code> → refresh once, then reject.	unit
L8	OAuth outage, or callback without <code>code</code>	<code>provider_unavailable</code> ; nothing stored.	emulator
L9	Auth, JWKS or PostgREST outage in <code>resolveSession</code>	<code>AuthError('unavailable')</code> → 503; never <code>null</code> .	unit
L10	Signed out globally, banned or deleted while holding a token	Rejected on the next Node-path request.	hosted
L11	<code>next</code> abuse	Falls back to the client default.	unit + emulator
L12	Staff of client A calls a client-B resource	<code>forbidden</code> .	unit + example consumer
L13	<code>updateUser</code> fails after recovery <code>verify0tp</code>	Stays on the reset screen with retry; <code>recovery_pending</code> blocks other kit pages until updated or signed out.	emulator + hosted
L14	A user whose permission comes only from an <code>mfa_required</code> role calls a protected route at <code>aall</code>	<code>mfa_required</code> ; the browser routes to challenge or enrol; after verification and session refresh, the retry succeeds.	unit + emulator + hosted
L15	Direct RPC to <code>auth_kit_private</code> , direct write to <code>memberships</code> , forged actor argument	All fail as anon, authenticated and wrong-client manager; the service role succeeds only through the exposed service-only wrappers.	hosted actor matrix
L16	Verified email equals a historical contact's email with no link row	The consumer returns no records.	example consumer
L17	<code>apply_model</code> rerun with an unchanged file	Empty diff, no model event, exit 0.	unit + hosted
L18	Model removes a role that users hold, or a permission still mapped elsewhere	Refused with the offending keys and holders; nothing written.	unit + hosted
L19	Manager calls <code>grant_membership</code> with a <code>manages_members</code> target; operator calls <code>bootstrap_manager</code>	Manager: <code>forbidden</code> , no event. Operator: granted, one event.	hosted actor matrix

Id	Case	Required outcome	Test
L20	User holds <code>buyer</code> and the <code>mfa_required</code> agent role at <code>aal1</code>	Permissions are the buyer's only; <code>mfaPending = true</code> ; <code>explain('documents:download:any')</code> shows agent withheld.	unit + emulator + hosted
L21	Zero state: join before the client exists, join before a model, bootstrap before a model, join on a closed client	<code>unknown_client</code> , <code>no_default_role</code> , <code>unknown_role</code> , <code>closed</code> ; no rows written; the browser shows <code>setup_pending</code> , or <code>no_access</code> for <code>closed</code> .	emulator + hosted
L22	Two operators apply different models concurrently	Serialized by the advisory lock; two events; <code>doctor</code> reports a file-hash mismatch for the operator whose file no longer matches the applied model.	hosted
L23	<code>revoke_manager</code> on the last manager	Refused; membership intact.	hosted
L24	Consumer checks a permission key absent from the model	<code>forbidden</code> ; <code>explain</code> shows no grantor; <code>doctor</code> lists keys referenced by the example consumer but not declared.	unit
L25	User signs out (or is banned) while holding an unexpired access token; the same token is used on (a) a Node endpoint and (b) a direct PostgREST read under a <code>has_permission</code> policy	(a) <code>invalid_token</code> , 401 on the very next request; (b) rows still returned until the token expires, then 401. Both outcomes are recorded side by side with the token lifetime.	hosted
L26	User holds <code>buyer</code> and the MFA-required agent role at <code>aal1</code> ; requests (i) another buyer's order and (ii) an order they are linked to, each (a) through the Node guard in section 4.3 and (b) through the RLS policy in section 4.6	(i): (a) 403 <code>forbidden</code> , <code>explain('orders:read:any')</code> shows agent withheld; (b) zero rows. (ii): (a) 200 via <code>orders:read:own</code> plus ownership; (b) the row. After <code>aal2</code> , all four succeed.	unit + hosted
L27	Enrollment matrix: first join; retried join; revoke by manager then sign in; two concurrent first joins; a join that fails after the enrollment insert (injected); the model later adds a new <code>self_assignable</code> role; a manager re-grants the revoked role	First: <code>enrolled</code> , one row, one event per role. Retry: <code>already_enrolled</code> , nothing written. After revoke: <code>already_enrolled</code> , membership stays absent. Concurrent: one enrollment row, one event per role in total. Failed join: no enrollment row; the next join enrolls. New role: enrolled users unchanged; the next new user gets both. Re-grant: membership back via a <code>manager</code> event.	hosted (concurrency and injected failure also in the emulator)
L28	<code>has_permission</code> and <code>has_role</code> called from a consumer policy as anon, enrolled member, member of another client, MFA-pending staff, manager at <code>aal2</code> , and for an unknown key; plus the private implementations addressed directly	Anon <code>false</code> without error; other client <code>false</code> ; MFA-pending staff <code>false</code> for staff-only keys and <code>true</code> for buyer keys; manager <code>true</code> ; unknown key <code>false</code> . HTTP : an RPC to <code>has_permission_impl</code> with the private schema selected is refused by PostgREST as an unexposed schema for both anon and authenticated. SQL impersonation (<code>set role</code> with <code>request.jwt.claims</code>): authenticated may execute the helper implementations and the five user-callable implementations and nothing else; anon gets <code>permission denied</code> on every private function; <code>doctor</code> shows <code>PUBLIC</code> has no <code>EXECUTE</code> anywhere.	hosted actor matrix
L29	On a live client: the model removes <code>manages_members</code> from the only held manager role; the model sets <code>manages_members</code> on a role with holders	Both refused with the affected holders; nothing written. The same models are accepted on a <code>registered</code> client with no holders.	unit + hosted

Id	Case	Required outcome	Test
L30	Every request-bearing command, called first with (i) a mutating payload and (ii) a no-op payload (unchanged model for <code>apply_model</code> , already a member for <code>bootstrap_manager</code> and <code>grant_membership</code> , not a member for the revokes), then retried with the same <code>request_id</code> and (a) the same payload, (b) a different payload	First call: (i) one event row and one <code>request_log</code> row; (ii) no event row and one <code>request_log</code> row with the no-op result. (a) The stored result, no new rows. (b) <code>request_conflict</code> , no new rows, in both (i) and (ii). A manager reusing another manager's id gets <code>request_conflict</code> . <code>join_client</code> and <code>register_client</code> have no <code>request_log</code> row and are idempotent on their natural keys.	unit + hosted
L31	User manages client A and is a buyer of client B; a server configured for B resolves the session	<code>memberships</code> contains only B's membership; <code>access</code> is B's; no PostgreSQL call mentions A.	unit + hosted
L32	<code>bootstrap-manager --email</code> with zero or two confirmed matches in a complete listing; <code>--email</code> when the page cap is reached before the end of the listing; <code>--user-id</code> for an unconfirmed user	Refused with <code>unknown_user</code> , <code>ambiguous_user</code> , <code>lookup_incomplete</code> or <code>email_unverified</code> ; nothing written; the <code>lookup_incomplete</code> message names <code>--user-id</code> . <code>--user-id</code> with a confirmed user succeeds and sets <code>state = live</code> .	unit + hosted
L33	<code>doctor</code> with only the secret key; <code>doctor --probe</code> with a disposable user	The first reports catalog grants and states that no actor probe ran; the second also reports real <code>anon</code> and <code>authenticated</code> outcomes; a deliberately widened grant is caught by both.	hosted
L34	Manager authority with two manager roles: role A <code>manages_members</code> without <code>mfa_required</code> , role B with it; U1 holds A and B, U2 only B, U3 only A; each calls <code>grant_membership</code> at <code>aal1</code> , then U2 at <code>aal2</code>	U1: granted via A (<code>has_role(A) true</code> , <code>has_role(B) false</code> , <code>explain</code> shows B withheld). U2 at <code>aal1</code> : <code>mfa_required</code> , nothing written. U3: granted. U2 at <code>aal2</code> : granted. A user with no manager role: <code>forbidden</code> . The JS evaluation fixture gives the same outcomes.	unit + hosted actor matrix

Id	Case	Required outcome	Test
L35	MFA reset: (a) user U with factor F and request R, run to completion; (b) retry U/R after success; (c) another user U2 with the same R; (d) two concurrent U/R runs; (e) crash after deleting the first of two factors, retry inside the lease, then after it; (f) crash before <code>mfa_reset_note</code> , retry after the lease; (g) crash as in (e), then two retries within a second of each other just after the lease; (h) the original runner pauses past the lease after deleting one of two factors while a second runner takes over and completes, then the original wakes and calls <code>deleteFactor</code> , <code>note</code> and <code>finish</code> ; (i) with a fake clock the claim age reaches 101 s between <code>listFactors</code> and the first <code>deleteFactor</code> , then the run is retried	(a) A pending row exists before <code>listFactors</code> , then completed with <code>factors_seen = [F]</code> , <code>factors_deleted = [F]</code> , <code>result = reset</code> , one event with <code>client_id</code> null. (b) The stored result, no admin API call , nothing written. (c) <code>request_conflict</code> before any admin API call; U2's factors untouched. (d) One run proceeds, the other gets <code>request_in_progress</code> before any admin API call. (e) Inside the lease <code>request_in_progress</code> ; after it <code>resume</code> returns <code>factors_seen = [F1, F2]</code> , the surviving factor is deleted, F1's not-found counts as deleted, finish stores both lists and <code>result = reset</code> ; no row ever records an empty <code>factors_seen</code> for a user who had factors. (f) <code>resume</code> with <code>factors_seen</code> null; the run lists the still-complete factors, notes them and completes. (g) The first <code>begin</code> gets <code>resume</code> and a fresh token, the second <code>request_in_progress</code> before any admin API call; one runner enters Auth; one completed row, one event. (h) The takeover completes with <code>result = reset</code> ; the original's late <code>deleteFactor</code> sees not-found; its <code>note</code> and <code>finish</code> each raise <code>run_superseded</code> (not <code>request_conflict</code> , since the token is checked before the state) and write nothing; exactly one event and one completed row carrying the takeover's token. (i) The CLI stops with <code>lease_expired</code> before the next admin API call; the row stays pending with <code>factors_seen</code> intact; the retry resumes with the original list and completes with one event.	unit (mocked admin API, fake clock for (i)) + hosted for (a)–(d)

10. Install and build contract

Supported hosts in 0.x: (a) a static site with its own build step and a conventional web server, and (b) a Node 22+ server for private APIs.

1. `npm install` a pinned version. `npx auth-kit init` writes a configuration skeleton, a model skeleton and `.env.example` with no values.
2. `npx auth-kit migrate` (Management API token from the environment), or paste the numbered SQL files into the SQL editor in order. `npx auth-kit doctor` verifies schema version, exposed schemas, grants (catalog mode), asymmetric signing keys and the redirect allow-list; `doctor --probe --probe-email <disposable user>` adds a real actor probe.
3. Configure the Supabase project: Site URL, exact redirect URLs, providers, SMTP, access-token lifetime, TOTP. This is manual, per project; see the [manual](#).
4. `npx auth-kit register-client`, `npx auth-kit apply-model --dry-run`, then `apply-model`, then `npx auth-kit bootstrap-manager --user-id <uuid>`. After any SQL editor change: `npx auth-kit export-model > auth-model.json`, review, commit.
5. Browser: import the ESM entry and its stylesheet (all classes prefixed `ak-`), pass the validated configuration, and use the default screens or the headless controller.
6. Server: `createAuthServer(config)`, `resolveSession` per request, `requirePermission`, then your own ownership rules. Sensitive data only on this path.
7. `npx auth-kit doctor --origin <url>` checks routes, CSP, `noindex` and the redirect allow-list against the project.

Not promised: automatic protection of an existing application's routes or schema, framework integrations beyond the two supported hosts, cookie-based sessions, row-level authorization, an admin UI for the model, and one-request revocation on the direct RLS path.

11. Provider setup

Each deployment needs, per environment:

- a **Supabase project** (Free plan or above) with asymmetric JWT signing keys, publishable and secret keys, a Management API personal access token for the operator, email confirmations on, exact Site URL and redirect URLs, a 30-minute access-token lifetime, email templates using `{{ .TokenHash }}`, and TOTP enabled if any role requires MFA;
- a **Google OAuth client** (web application) if Google sign-in is enabled, with the deployment's origins and the redirect URI `https://<project-ref>.supabase.co/auth/v1/callback`; its client id and secret are entered in the Supabase dashboard;
- an **SMTP provider** and sending domain with SPF and DKIM. The operator chooses and configures it per deployment. Supabase's built-in sender is rate limited and meant for development only.

Secrets go in a local environment file outside any repository.

12. Limitations

- Supabase Auth only; no other identity back end.
- Two supported host types (section 10). Other frameworks are possible but untested.
- Type-level RBAC only: no relationship- or attribute-based authorization. Row ownership is yours.
- On the direct RLS path, Auth session revocation is bounded by access-token expiry.
- No member listing in 0.x. A consumer that needs a staff directory fills it from resolved sessions.
- No admin UI for the model and no cookie-session mode.
- The project's secret key is root for the project.
- Supabase project configuration is manual per environment.
- Write throughput per client is serialised by one lock.